

# Notary

The context lie detector: it cross-examines a data catalog's claims against measured reality and writes the verdicts back.

DataHub Agent Hackathon  
Category 1: Agents That Do Real Work  
notary-replay.vercel.app  
github.com/OrionArchitekton/notary

## The problem

A column says "transaction amount in USD" but stores integer cents. An agent grounded on that catalog can put a revenue calculation 100x off, with the catalog's own authority behind it. Catalogs accumulate claims about units, freshness, completeness, enums, and deprecation, and nothing checks whether they are still true. As agents inherit catalogs as ground truth, a stale claim stops being an annoyance and becomes an amplifier.

12

catalog lies planted across 5 claim types in the seeded demo warehouse

9

caught with evidence; every miss declared with its reason, never hidden

0

false positives across the truthful controls

119

tests, incl. live DataHub round trips and a pin that fails if the published table drifts

## What it does

|            |                                                                                                                                                                                                                                                                                                                                                           |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| READ       | Reads the asset's live descriptions from DataHub (GraphQL), consuming what the catalog actually says right now.                                                                                                                                                                                                                                           |
| EXTRACT    | Claude extracts typed, checkable claims behind strict entailment gates: a claim the description does not actually state never reaches a probe. Failed extractions are disclosed and left unverified, never guessed at.                                                                                                                                    |
| PROBE      | Deterministic, bounded, read-only SQL against the warehouse. Scans are capped; universal claims get a verdict only from complete scans.                                                                                                                                                                                                                   |
| ADJUDICATE | A pure three-way rubric per claim type: CONFIRMED, CONTRADICTED, or UNVERIFIABLE, fail-closed, every verdict carrying its evidence.                                                                                                                                                                                                                       |
| WRITE BACK | Through the DataHub MCP Server: a trust ledger (structured properties), an evidence dossier per adjudicated claim (documents), and a provenance-labeled corrected description. A contradicted unit claim on an asset with recorded query traffic also raises an operational incident (GraphQL), and that danger gate fails closed without usage evidence. |

## The differentiator

**The verdict lives where the next reader looks.** Humans see the correction and the incident in the DataHub UI; agents inherit the trust ledger through the stock MCP read tools. We demonstrate the flip end to end: a minimal catalog-grounded agent reading only through stock MCP tools relays the unverified USD claim when the ledger is withheld, and refuses it, quoting the measured cents evidence, when the ledger is present. Both answers are captured and shown side by side on the replay page.

**And the scorecard is honest.** The evaluation table is published verbatim, misses included, and regenerated by one command; a test fails if the README table drifts from the command's output.

# Engineering rigor

**Deterministic where stability matters.** Captured Claude completions replay verbatim (prompt-key bound; a fixture copied under another key, drifted from its prompt, or stale relative to the fresh evidence aborts the build). The warehouse rebuilds from a fixed seed; probes and rubrics are pure. Two replay-data builds are byte-identical, and the committed payload the host serves is byte-compared to a fresh capture in the suite.

**Fail-closed as a discipline.** Fixture-store completeness and unexpected extraction failures abort before anything is published (exit 2/3). The incident gate requires catalog usage evidence and refuses to fire without it. UNVERIFIABLE is a first-class verdict, never hidden.

**Bounded probes.** Read-only connections, capped scans, and a complete-scan bar for every universal claim, including both verdicts of the flagship USD rubric.

**Reviewed adversarially, and it drew blood.** Two independent engines (an internal adversarial fleet and Codex) refuted every judge-facing sentence against the code, tests, git history, and live site. All findings were fixed, several against our own favorite claims:

*The percent rubric we first shipped was **killed by our own review pipeline**: a stored fraction and legitimate sub-1-percent values are indistinguishable by distribution alone, so contradiction is now unreachable by design.*

*The USD rubric could issue a verdict from capped-scan prefix statistics; review forced it to **require a complete scan for both verdicts**, with regression tests.*

*The demo narration claimed the pre-ledger agent "repeats the lie"; our own captured answer showed it hedging instead, so **the copy was rewritten to match the evidence**.*

## Challenges

Extraction gating is where honesty is won or lost: entailment checks (token-aware, negation-aware, closed-set) keep Claude from probing claims the description never made. One capture prompt is deterministically refused by the provider's content filter; it is declared, scored fail-closed, and disclosed in the published table rather than dropped. Incident idempotency rides on search-index eventual consistency, so raise-or-reuse is resolved by a stable title lookup with the consistency window documented.

## The honest scorecard

| claim type        | planted lies | caught | missed | controls | false positives |
|-------------------|--------------|--------|--------|----------|-----------------|
| unit_scale        | 4            | 1      | 3      | 2        | 0               |
| freshness         | 2            | 2      | 0      | 0        | 0               |
| completeness      | 3            | 3      | 0      | 0        | 0               |
| domain_enum       | 2            | 2      | 0      | 2        | 0               |
| deprecation_usage | 1            | 1      | 0      | 0        | 0               |
| total             | 12           | 9      | 3      | 4        | 0               |

The three unit misses are declared, not pending: 0-to-1 distributions are scale-ambiguous by design, and magnitude bands for ms or grams would be domain guesses. This project does not guess.

### TRY IT IN 60 SECONDS

1. Open **notary-replay.vercel.app**: the frozen, reproducible replay of the recorded run, disclosed on-page.
2. Expand the flagship evidence dossier: the probe SQL and measurements, exactly as Notary writes them to DataHub.
3. Read the two captured agent answers side by side: the same catalog, with and without Notary's trust ledger.

python

duckdb

datahub oss

mcp-server-datahub

claude-opus-4-8

graphql

structured properties

playwright

vercel

apache-2.0